

An FPGA based Tiled Systolic Array Generator to Accelerate CNNs

Veerendra S Devaraddi and Nanditha Rao

International Institute of Information Technology Bangalore, India

Email: {veerendra.s, nanditha.rao}@iiitb.ac.in

Abstract—The main computation in any CNN is convolution operation. This computation shows significant potential for massively parallel implementations on an FPGA. Systolic arrays with their intrinsic pipelining have been explored for CNN inference. In this paper, we present a systolic array architecture suitably designed for a novel method of convolution operation. We implement an image-kernel convolution and test it with representative image inputs to several models like LeNet-5, AlexNet, VGG-16, and Resnet-34. We compare the proposed design with conventional convolution and HLS based designs. We limit our implementation to resource constrained FPGA: AMD-Xilinx Zynq 7020 platform. We observe that the proposed architecture outperforms the direct convolution method and HLS pipelined designs by $2\times$ and $2.1\times$, respectively, on average. Since DSP blocks are scarce resources, we constrain our implementation to avoid DSP blocks and use the LUTs instead. Thus, our implementation uses nearly $9\times$ more LUTs than baseline convolution but $8\times$ fewer LUTs than the HLS pipelined implementation. We further accelerate the convolution throughput by $11\times$. We achieve this by implementing a tiled systolic architecture that completely utilises the parallel computing resources of the FPGA.

Index Terms—Convolutional neural networks (CNNs), FPGA, systolic array, hardware accelerators

I. INTRODUCTION

In deep learning, a convolutional neural network (CNN) is one of the key algorithms commonly applied to image and video classifications. The key mathematical computation in a CNN is the convolution operation, a dot product of the layer's input feature map (image), and a convolution kernel to produce an output feature map. A typical CNN model consists of layers such as convolutional layer (CONV), max-pooling, rectified linear unit (ReLU), and fully connected (FC) layers. There are several popular CNN architectures, such as the LeNet-5, AlexNet, VGGNet, and Resnet-34, with a different convolutional (CONV) layers and different sizes of the input feature maps, and varying filter sizes. These computations in every layer are computationally intensive but also offer significant opportunities for massive parallelization. Due to the inherent parallel structures available in an FPGA and its reconfigurability, we have seen an increased interest in FPGA implementations of CNNs [1-4]. However, to fully utilise the parallel computation capability of an FPGA, a suitable architecture for reconfigurable and scalable CNNs is necessary.

A hardware implementation of CNN layers offers the following challenges: computational complexity in the CONV layers, handling data movement between the multiply and accumulate (MAC) units and managing a large amount of input

and output data in memory [5], [6]. These tasks become more challenging on resource constrained FPGA devices, especially for large models [7] [8]. A few existing architectures for CNN implementations on FPGA are fused-layer architecture, layer specific architecture, convolution optimisation architectures, and systolic array architectures. [9] employs fused-layer architecture for acceleration and reduces off-chip data transfer significantly. [10] employs layer specific architecture, which uses convolution layer engines to achieve high throughput, but they suffer from high resource utilisation. Convolution optimisation architectures employ techniques such as the Winograd algorithm and GEMM based algorithms to accelerate convolution operations, which are explored by [4], [11], [12], and [13]. Systolic array architectures are known for their high throughput and minimal memory access, and it is explored by [3] [14] [15] and [16].

A systolic array is a specialised architecture that consists of an array of repetitive pipelined processing elements (PE). The regular layout of PEs when implemented on an FPGA, helps perform parallel computing and significantly reduces the time for the computation. As a result, systolic arrays have been widely explored to accelerate the matrix MAC and multiplication operations. Typically, the convolution operations are performed using the conventional element-wise multiplication, then summing the results to obtain the output. Most existing works [4] [17] exploit the inherent parallelism in MAC operations that dominate the computations. However, alternate methods to perform convolution with a reduced number of matrix multiplications are explored in this paper.

In this paper, we propose a novel convolution methodology based on matrix multiplication followed by diagonal addition (MMDA) as against the dot-product based conventional approach. With the proposed approach, during an image-kernel convolution, we can reduce the number of horizontal strides needed by the kernel, leading to a reduced number of kernel-image matrix operations by K , where K is the size of the kernel, as compared to the conventional method of convolution. To accelerate the convolution operation, we propose to use a specific systolic structure that consists of an array of $(2\times K-1)\times(2\times K-1)$ PE/DEs where each PE performs a multiply and add operation. The elements of matrix multiplication result exit the systolic array through the diagonal wires. We then add the diagonal elements of the resultant matrix to get the convolution results as per our proposed MMDA approach. Furthermore, the output data flow in this structure synchronises with the proposed convolution method such that we do not need to

buffer the data for processing.

One of the key challenges for the proper operation of a systolic array is to sequence the data correctly at the inputs of a systolic array and synchronise the PEs at every clock cycle. In addition, We also need to synchronise the output data flow to perform the diagonal addition in the correct clock cycles. To assist this task, we design the following control units: Data Conversion Unit (DCU), Zero Padding Unit (ZPU), and Convolution Compute Unit (CCU). The implementation supports integer datatype and can be extended to float or fixed datatype.

To further accelerate the convolution, we instantiate multiple systolic array tiles (SATs) such that we fully utilise the available FPGA resources. We refer to this architecture as tiled systolic architecture (TSA). We divide the input image into horizontal blocks and map them onto different systolic tiles to perform each computations in parallel. We automate this systolic tile generation such that, given the resource constraints (LUTs, DSPs, block RAMs, etc.) of a particular FPGA, we can estimate the maximum number of tiles that can nearly use the entire available resources and achieve the best acceleration. We implement the following five different designs for comparison: systolic array with conventional convolution, the systolic array with MMDA-based convolution, TSA architecture, HLS based non-pipelined method, and HLS based pipelined method of convolution. We compare the performance and resource utilisation of these designs on representative dimensions of images and kernels from the following model architectures: Lenet-5, FSR-CNN, AlexNet, Resnet-34 and VGG-16 architectures.

In this paper, we limit our work mainly to resource constrained FPGAs to demonstrate the maximum throughput that can be achieved on such platforms. We implement all the designs on AMD-Xilinx Zynq based Zedboard using Vivado 2019.1 and High Level Synthesis (HLS) tools. We observe that the proposed MMDA-based systolic architecture outperforms the conventional convolution based systolic structure and HLS pipelined designs by $2\times$ and $2.1\times$, respectively, on average. Our systolic based approach outperforms the pipelined implementation generated through HLS in terms of both throughput and resource utilisation. Since we do not use any DSPs in the design (see Section V-C), we end up using $9\times$ more LUTs compared to the conventional convolution approach. However, our implementation still uses nearly $8\times$ lesser LUTs than the HLS pipelined implementation. The tiled systolic architecture further improves the performance by $11\times$ over the baseline MMDA-based systolic design on the mentioned FPGA. This throughput increases by using larger FPGAs with more resources. To demonstrate this, we also synthesise our design on ZCU104 and observe that our implementation's throughput is more competitive when compared with existing work.

The rest of this paper is organised as follows: Section II explains the proposed convolution method mathematically and gives a visual intuition of the method. Section III explains the systolic array architecture and data flow. Next, We explain the tiled systolic architecture in Section IV. Section V presents the different designs' latency, throughput, and resource utilisation. Finally, we conclude the paper in Section VI.

II. PROPOSED CONVOLUTION METHODOLOGY

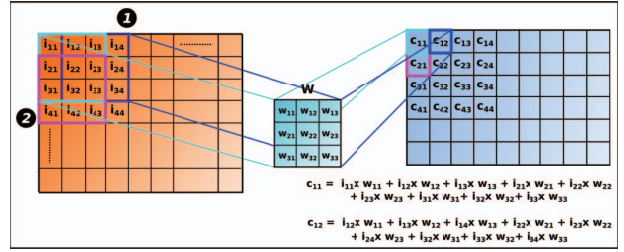


Fig. 1: Diagram of kernel sliding over an image during conventional convolution method

Consider a 2D input image I of size $R \times C$, in which R is the total number of rows and C is the number of columns in the image. Assume a kernel W of size $K \times K$ consisting of weights and a total of N kernels. To perform a convolution between the image and the kernel, we typically perform an element-wise multiplication between the $K \times K$ part of the image with the kernel and sum up the products to obtain the first convolution output in the resultant output matrix. We call this element-wise multiplication followed by addition (EWMA). The kernel will then slide over the input image to the right by a stride of one, as shown in Step (1) in Figure 1. Let us call this the column stride. This process continues till the kernel reaches the last column. The kernel is then moved back to the first column and down by one row, as shown in Step (2) in Figure 1. Let us call this the row stride. The element-wise multiplication and summation continue till we reach the end of the image. So, there will be $R \times (C-K+1)$ element-wise multiplications in this method.

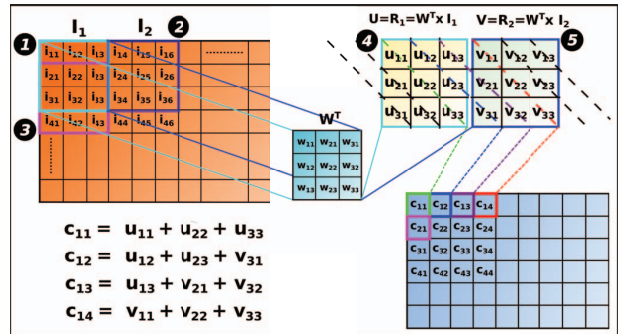


Fig. 2: Kernel sliding over an image in the proposed MMDA method

We illustrate the proposed convolution method in Figure 2 and explain it in detail below:

- 1) Transpose the kernel matrix W .
- 2) Place the kernel of size $K \times K$ onto the top-left corner of the input image matrix I_j . Instead of an element-wise multiplication, perform a matrix multiplication between the input image matrix and transposed kernel. Save the resultant output matrix R_j . For the convenience of

representation, we depict this matrix as U in Step (4) of Figure 2.

$$U = R_j = W^T \times I_j$$

where R is the resultant matrix, W is the kernel, and I_j is the j^{th} $K \times K$ matrix from the input image. The input image I is padded with zeroes to convert the width into a multiple of K .

- 3) Slide the kernel by a stride of “ K ” to the right over the input image, as shown in Step (2) in Figure 2. For example, if the kernel is of size 3×3 , slide the kernel by three columns on the input image. Recall that in the traditional convolution method, the kernel slides by a column stride of one. Next, repeat the matrix multiplication operation and save the resultant output matrix elements in, say, matrix V .
- 4) Once the kernel reaches the last column on the input image I , reset its position to the first column and slide it one row down as shown in Step (3) in Figure 2. Repeat the process of matrix multiplication by moving the kernel left to right (in column strides of K) and top to bottom (in row strides of one) till the entire input image is processed.
- 5) Elements of the resultant output matrices are diagonally added to generate the final convolution results, as indicated by the dashed lines in steps (4) and (5) in Figure 2. The matrix multiplied outputs embed multiple element-wise multiplication and addition components; therefore it only needs a horizontal stride of K instead of one. This reduces the number of multiplications needed in the proposed method.

These steps can be represented as follows: Let W and I be the kernel and image matrices of size 3×3 .

$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{bmatrix} I_j = \begin{bmatrix} i_{11} & i_{12} & i_{13} \\ i_{21} & i_{22} & i_{23} \\ i_{31} & i_{32} & i_{33} \end{bmatrix}$$

Let U and V be the first two resultant matrices after matrix multiplication, and be defined as follows:

$$U = R_1 = W^T \times I_1, V = R_2 = W^T \times I_2$$

where I_1 and I_2 are the parts of the image as shown in (1) and (2) in Figure 2. Let us represent,

$$U = \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ u_{21} & u_{22} & u_{23} \\ u_{31} & u_{32} & u_{33} \end{bmatrix} V = \begin{bmatrix} v_{11} & v_{12} & v_{13} \\ v_{21} & v_{22} & v_{23} \\ v_{31} & v_{32} & v_{33} \end{bmatrix}$$

Let C be the desired final convolution matrix C after the convolution of Image I with kernel W , whose first row is shown below.

$$C = [c_{11} \quad c_{12} \quad c_{13} \quad \dots]$$

The elements of the convolution matrix C is obtained by adding the diagonal elements of matrices U and V , and is illustrated by the dashed lines in Figure 2:

$$c_{11} = u_{11} + u_{22} + v_{33}$$

$$c_{12} = u_{12} + u_{23} + v_{31}$$

$$c_{13} = u_{13} + v_{21} + v_{32} \text{ and so on.}$$

Similarly, further convolution results can be calculated. We call this matrix multiplication and diagonal addition (MMDA) method of convolution. It equivalently produces K convolution results using one matrix multiplication result (which takes K cycles). Therefore one convolution result is produced in every clock cycle. The advantage of the proposed method is as follows: the horizontal stride during the matrix multiplication is K instead of 1. Therefore the number of matrix operations in the proposed method is reduced by a factor of K compared to the conventional EWMA method. When applied to a convolution layer of a CNN, this method can significantly improve the model's performance.

On the other hand, several intermediate matrices U , V , and so on produced as a result of the matrix multiplications (before we sum up the diagonals). However, this is not an overhead, as the summation of the diagonal elements can take place as soon as they are generated and in parallel with the next matrix multiplication operation. In Section III-B, we will demonstrate that this proposed convolution approach synchronises with the systolic structure's output data flow and thus, avoids the need to buffer the intermediate matrices.

III. SYSTOLIC ARRAY GENERATOR

In this section, we describe an implementation of the proposed MMDA convolution operation using a systolic array architecture. We propose a systolic array generator that generates the structure of the systolic array for a given kernel size. We present the following major components of the systolic array:

- 1) The architecture of the systolic array, along with its processing elements (PE).
- 2) The computation in each PE.
- 3) The dataflow model of the two input matrices: image I_j , kernel W , and the resultant output matrix R_j .
- 4) A data conversion unit (DCU) and Zero padding unit (ZPU) to transform the input image into the required format suitable for systolic execution.
- 5) A convolution compute unit (CCU) and a controller that manages the diagonal addition and pipelined data movement in the systolic array, respectively.
- 6) Memory management from an FPGA implementation perspective.

We present the entire architecture of the systolic array in the block diagram in Figure 3a.

A. Systolic array structure and Data flow

We initially considered a 3×3 systolic structure to perform a 3×3 matrix multiplication [2]. It had the following disadvantages over the proposed systolic array: The matrix multiplication result elements get accumulated in the accumulator registers of each PE. So, we need to reset the accumulators after every matrix multiplication. Furthermore, the accumulator registers produce results that require us to buffer halves of two resulting matrices, resulting in a more complex controller design. Therefore, we choose to implement

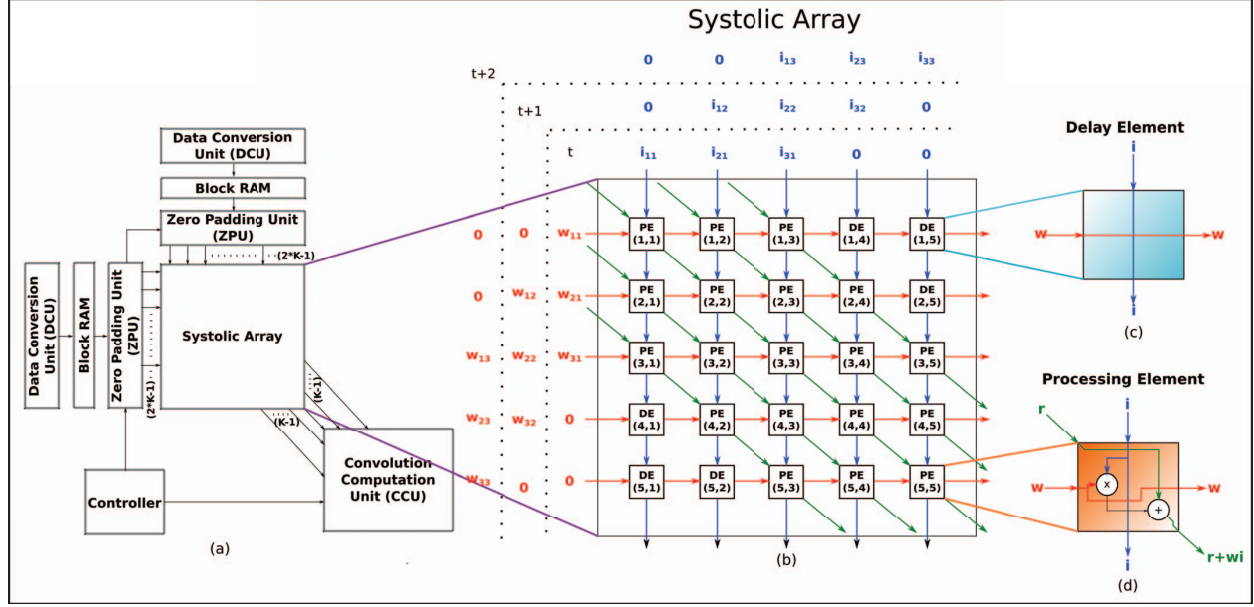


Fig. 3: Implementation of systolic array architecture. (a) Block Diagram (b) Systolic array showing initial inputs to PEs. Diagonal inputs of top-left PEs are always zero. (c) Delay element structure (d) Processing element structure

a different systolic structure of size $(2 \times K - 1) \times (2 \times K - 1)$, which has a simpler controller at no extra latency cost.

We implement a systolic array consisting of processing elements (PE) and delay elements (DE), as shown in Figure 3b. Figure 3d shows the PE. Each PE performs a multiplication of the vertical input (image elements i) with the horizontal input (kernel elements w) and adds it with the diagonal input (r) in every clock cycle t . This indicates that it performs multiplication, followed by an addition operation every clock cycle, and passes the output on the diagonal output wire. It also transfers the horizontal and vertical inputs to their respective outputs at every clock cycle. This can be represented by the equation:

$$r_{t+1} = w \times i + r_t$$

$$w_{t+1} = w_t \text{ and } i_{t+1} = i_t$$

where the subscript t represents the t^{th} clock cycle.

Each Delay Element (DE) in the structure produces the required number of delays before the inputs reach the PEs. Hence it just transfers the horizontal and vertical values to their respective outputs at every clock cycle, as indicated in Figure 3c. This can be represented by the equation:

$$w_{t+1} = w_t \text{ and } i_{t+1} = i_t$$

The size of the systolic array structure depends only on the size of the kernel and not on the size of the image. So, if $K \times K$ is the size of the kernel, then the systolic array has a size of $(2K-1) \times (2K-1)$. For instance, if kernel size is 3×3 , we need a 5×5 systolic structure, as shown in Figure 3b. Throughout this paper, we assume a kernel of size 3×3 for our discussion. Since we are mapping a 3×3 image and kernel onto a 5 PE/DE structure, we need to align inputs and kernels to 3 PE/DEs

and pad zeroes to the other 2 PE/DEs before we can start the systolic execution. The initial sequence of inputs at the PEs in clock cycle t are $\{w_{11}, w_{21}, w_{31}, 0, 0\}$ and $\{i_{11}, i_{21}, i_{31}, 0, 0\}$ as shown in the diagram. In the next clock cycle ($t+1$), the inputs are $\{0, w_{12}, w_{22}, w_{32}, 0\}$ and $\{0, i_{12}, i_{22}, i_{32}, 0\}$, and so on, as shown in Figure 3b. Note that the diagonal inputs (green inputs in Figure 3b) at the top-left corner of the systolic array are always zero. Outputs are calculated as it propagates along the diagonal wires in the systolic array at every clock cycle.

Note that the key advantage of systolic structures is the reduced frequency of memory access. Data is only fed to the top row and left column of PEs; the rest of the PEs get data from their neighboring PEs (without needing to access memory). In Figure 4, we show the status of the systolic array at the end of clock cycles 5, 6, and 7. At the end of clock cycle 5, the first elements of the resultant matrix multiplication outputs (u_{11}, u_{12}, u_{13} , and so on) begin to exit the systolic array at the bottom right corner. We use these matrix multiplication results to produce convolution as per the MMDA method (described in the next section). By the end of clock cycle 7, we complete one matrix multiplication. The advantage of this systolic structure is that it does not accumulate the results in its PE/DEs but outputs the results at every clock cycle.

B. Convolution Compute Unit

As shown in Figure 4, the matrix multiplication results flow out in a particular pattern along the diagonal wires D1, D2, D3, and so on. As discussed in section II, the MMDA method involves diagonal elements addition from two adjacent output matrices U and V , to produce the convolution results. After the first matrix multiplication results start exiting the systolic

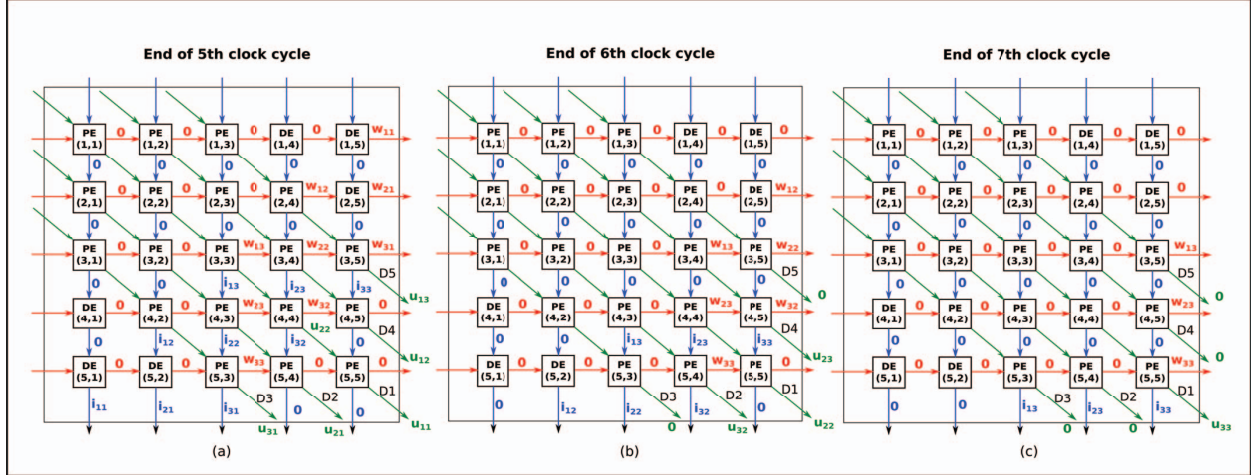


Fig. 4: Systolic execution illustrating the inputs and outputs of all the PEs at the end of clock cycles 5, 6, and 7, respectively

array at the bottom right corner in clock cycle 5, the CCU begins to add the diagonal elements of the output matrices in the order described in Figure 2 to generate the convolved matrix. Out of the five output wires of the systolic array shown in Figure 3 (bottom-right green diagonal wires), we need to select the appropriate wires to perform the addition. This selection must be synchronised with the input data flow and added accordingly (summation is across multiple wires). The CCU does this entire synchronisation and addition of the diagonals to produce one convolution result in every clock cycle.

C. Data conversion unit and Zero padding unit

The computation in a systolic array can be represented in terms of three elements (x, y, t) , where (x, y) are the indices in the array (at a particular PE) in clock cycle t . The critical challenge in the systolic array is sequencing the data to the systolic structure's $2 \times K-1$ horizontal and $2 \times K-1$ vertical inputs. We implement a data conversion unit (DCU) in Python, which takes an image and a kernel of any dimension as inputs. It transforms or breaks down the image into matrices of size $K \times K$ with a horizontal stride of K and a vertical stride of one, where K is the size of the kernel. This transformed data is stored in block RAMs of the FPGA along with a valid bit which indicates the start and end of valid data. For example, if the size of the kernel is 3×3 , the input image is broken down into matrices of 3×3 (with row stride three and column stride one).

However, we have 5 PE/DEs that read in the input image (from the top) and 5 PE/DEs to read in the kernel (from the left), but the matrices are of size 3×3 . So, we need to align input and kernel elements to 3 Systolic Array (SA) elements and pad zeroes to the other 2 SA elements before we can start the execution. Therefore the block RAMs feed the zero padding unit (ZPU), which pads zeroes at appropriate positions at every clock cycle (implemented in hardware). This alignment is key to the successful operation of the entire

architecture. ZPU performs memory accesses and feeds the inputs of the PEs.

D. Controller design and Memory Management

The datapath in the systolic architecture consists of PEs and block RAMs. The input image and kernels are stored in block RAMs on the FPGA. There are several data dependencies during the systolic execution, and the controller must be pipeline-aware to manage the data flow. Figures 3 and 4 show the dependency. We design a top-level controller which performs the following tasks:

- 1) Connect the block RAMs with the PEs on the left/top edges.
- 2) Generate the memory addresses for the block RAMs.
- 3) Enable the CCU at the correct clock cycle to add the diagonal elements.

Overall, the controller manages the data movement from the memory to the systolic structure. Therefore, it is crucial to obtain the correct convolution outputs.

IV. TILED SYSTOLIC ARRAY

To further improve the throughput and use the parallel resources available on the FPGA, we instantiate multiple systolic array tiles (SAT). We refer to this architecture as tiled systolic architecture (TSA). Each tile (SAT) comprises of the systolic array architecture (SAA - which includes the CCU), as shown in Figure 5c. We design a parameterised model which automatically generates multiple SATs. We choose low-end resource constrained FPGAs such as the Zedboard to demonstrate the maximum throughput that can be achieved on such platforms. The maximum number of tiles that can be instantiated in an FPGA depends on the maximum resources that we wish to utilise. To utilise 90% of the LUT resources on Zedboard, we instantiate 15 SATs, as shown in Figure 5b. The inputs to each SAT are provided as follows: The 2D input image is divided into multiple horizontal blocks where each block consists of several $K \times K$ matrices to be convolved. For

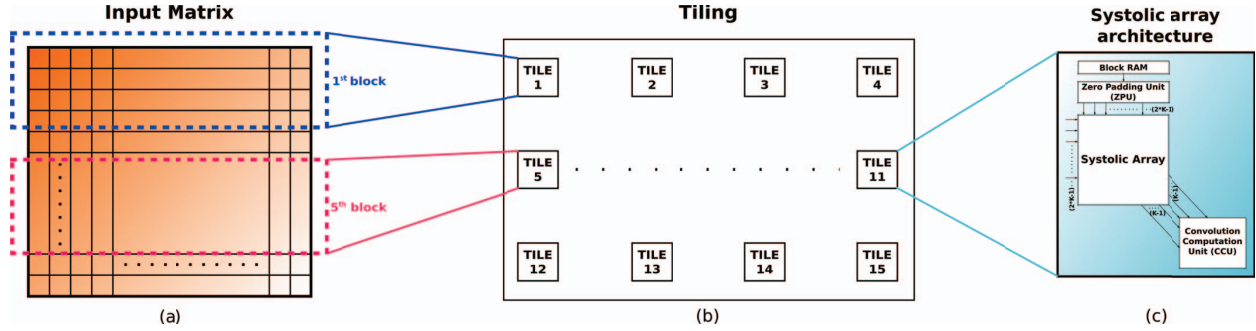


Fig. 5: Diagram showing the tiled systolic architecture. (a) An Input image is divided into blocks and assigned to SATs (b) Grid of SATs (c) The SAT made up of the systolic array with CCU

instance, an $R \times C$ image is divided into b blocks where b is the number of SATs instantiated. Each block is mapped onto a different SAT, as shown in Figure 5a, and the same input weights are mapped to all SATs. We divide the image into horizontal blocks to easily synchronise the diagonal additions needed to produce the convolved outputs as per our proposed MMDA method.

The original input image matrix is stored in the block RAM and then parallelly loaded on all the SATs' vertical inputs of the systolic array. Similarly, the weights in the kernel stored in the block RAM get loaded on all the SATs parallelly on the horizontal systolic inputs. For instance, the mapping can be represented as follows:

- 1) Tile 1 (SAT 1): Image 1, kernel 1, block 1
- 2) Tile 2 (SAT 2): Image 1, kernel 1, block 2
- 3) Tile 3 (SAT 3): Image 1, kernel 1, block 3, and so on.

This results in parallel processing across the image such that the throughput of the convolution increases by a factor of b compared to a single systolic structure, where b is the number of tiles. We automate this systolic tile generation such that, given the resource constraints (LUTs, DSPs, block RAMs, etc.) on a particular FPGA, we can estimate the maximum of SATs that can populate nearly the entire available resources. This method can be extended to other FPGAs with more resources to achieve higher throughput.

V. EXPERIMENTAL SETUP AND RESULTS

The proposed MMDA convolution method with the systolic architecture is implemented in Verilog. It is targeted at accelerating image-kernel convolutions in CNN models. In this work, we evaluate the approach on the resource constrained FPGA, Zynq 7020 (xc7z020clg484-1). We also synthesise our designs for ZCU104 to compare our throughput with other existing works. We consider the following designs to compare latency, throughput, and resource utilisation:

- 1) Design 1: The proposed MMDA convolution method integrated with systolic architecture.
- 2) Design 2: The conventional EWMA based convolution method on the systolic architecture. The element-wise matrix multiplication is done using the proposed systolic array in Figure 3b, but with a different input pattern.

3) Design 3: Tiled systolic architecture (TSA) as shown in Figure 5.

4) Design 4: The conventional EWMA based convolution implemented with HLS, without pipelining.

5) Design 5: The conventional EWMA based convolution implemented with HLS, with pipelining enabled.

We implemented these designs using Vivado 2019.1 and Vivado HLS. Designs 4 and 5 are estimates from Vivado HLS and are not tested on hardware. We consider representative image and kernel sizes from VGG-16, Resnet-34, LeNet-5, AlexNet, and FSRCNN as inputs to the designs. Block RAMs are used to store the image data in 16-bit integer format, as described in Section III-C. After the design is implemented on the FPGA, the convolved output data is probed using an integrated logic analyser (ILA).

A. Latency and throughput comparison

We compare the latency of a single image-kernel convolution for the five designs we implemented. Here, latency indicates the time taken from the first memory access till all the convolution outputs are produced. We consider several single-channel input images of various sizes to compare the latencies across the designs. These sizes correspond to the dimensions of convolution (CONV) layers of different CNN models. We present these results in Table I. We observe that design 1 (no DSPs) (proposed MMDA based systolic architecture) performs $2\times$ faster than design 2 on average. This is mainly because of the reduced number of multiplications in an image-kernel convolution compared to the conventional EWMA method. It is interesting to note that Design 1 (no DSPs) also outperforms design 5 by $2.1\times$. Design 3 further accelerates design 1 by $11\times$ due to the parallelism. We can easily extend this calculation to an entire CONV layer (with multiple kernels) to show the performance improvement.

With design 3, we are able to achieve 1617 Mega convolutions per second (Mconv/s). We test design 3 on Zedboard and synthesize it for ZCU104 FPGA board (post-implementation). We compare it with existing works (one of them is implemented on resourced constrained FPGAs) in Table III. The throughput is presented in terms of Giga operations per second (GOPS). We observe that our design is competitive with other implementations, even with tight resource constraints.

TABLE I. Latency comparison of a single image-kernel convolution in the CONV layers of different CNN models. We use a 3×3 kernel in all models. Here, P indicates pipelined, NP indicates non-pipelined design

Image sample	Image size	Design 1: MMDA		Design 2: EWMA	Design 3: TSA(15)	Design 4: EWMA	Design 5: EWMA
		(DSPs)(us)	(no DSPs)(us)	(us)	(us)	HLS(NP)(us)	HLS(P)(us)
LeNet-5 layer 1	28×28	3.851	5.638	10.469	0.557	280	11
AlexNet layer 3	256×256	319.063	467.120	930.021	40.795	28392	986
FSRCNN layer 1	144×176	122.406	179.233	357.489	16.422	10875	377
VGG-16 layer 1	224×224	243.210	356.121	711.260	29.226	21689	755
VGG-16 layer 1	112×112	61.098	89.463	176.256	8.461	5626	185
Resnet-34 layer 2	56×56	15.037	22.018	43.306	2.115	1284	45.7

TABLE II. Resource utilisation on the FPGA. Here, P indicates pipelined, NP indicates non-pipelined design

Resource	Design 1: MMDA		Design 2: EWMA	Design 3: TSA(15)	Design 4: EWMA	Design 5: EWMA	Available
	(DSPs)	(no DSPs)			HLS (NP)	HLS (P)	
LUT	430 (0.808%)	4335 (8.148%)	469 (0.881%)	44774 (84.161%)	221	32575	53200
FF	1008 (0.947%)	3531 (3.318%)	1145 (1.076%)	16036 (15.071%)	335	70407	106400
Block RAM	52.5(37.500%)	52 (37.142%)	52 (37.142%)	67.5 (48.214%)	0	0	140
DSP	19	0	0	0	3	369	220

TABLE III. Throughput comparison of proposed work with existing work

	Design 3: TSA	Design 3: TSA	[7]	[5]	[8]	[6]
FPGA	Zynq 7020	Zynq ZCU104	Zynq 7020	Virtex 7 VC709	Virtex 7 485T	Virtex 7 VX485T
Frequency (MHz)	107.782	150.015	100	156	-	100
LUTs	44774 (84.16%)	185.9K (80.70%)	40897 (76.87%)	273.8K(63%)	15285 (5.03%)	186K (61.3%)
FFs	16036 (15.07%)	64684 (14.04%)	21285 (20.00%)	-	2074 (0.34%)	205K (33.8%)
BRAMs	67.5 (48.21%)	67.5 (21.63%)	54 (38.57%)	1913 (65%)	571 (55.43%)	1024 (50%)
DSPs	0 (0%)	0 (0%)	115 (82.14%)	2144 (60%)	564 (20.14%)	2240 (80%)
Data type	integer	integer	-	fixed	fixed	float
Bit-width/Precision	16	16	8-16	16	16	32
Throughput(GOP/s)	68.12	379.24	2.25	565.94	6	61.62

The throughput can be further increased by implementing our design on larger FPGAs with more resources. To demonstrate this, we synthesize our design on ZCU104 FPGA and observe that throughput (post-implementation) is competitive with the existing works.

B. Frequency improvements: Critical path analysis

For design 1, we are able to achieve a frequency of 205.423 MHz when DSPs are used and 140.292 MHz when DSPs are not used on the Zynq 7020 FPGA. This does not change with an increase in bit-width (tested up to 16 bits) or the size of the image. In the case where DSPs are used, further frequency increase is limited by the ILA and the Block RAMs. In the case where DSPs are not used, further frequency increase is limited by the multiplication operation in PEs. As future work, we plan to increase the frequency by implementing a 2-stage pipeline in PEs so that we can perform multiply and add in different clock cycles.

In the case of TSA, the critical path is from block RAM till the end of multiplication operation logic (not DSP) in the PE. The total delay along this path is mainly contributed by the net delay from block ram to inputs of PE, and it significantly reduces the frequency. This is because TSA is implemented such that it fully utilises the FPGA resources, which makes some of the SATs get placed far from the block RAM. Therefore the performance improvements in Design 3 is only $11 \times$ when compared with Design 1 (no DSPs), instead of

$15 \times$. Few possible optimizations can be, to pipeline the path by inserting flip flops or duplicating the block RAM contents.

C. Resource utilisation

Table II shows the hardware resource utilisation for the implemented designs on the same FPGA platform. We constrained the design to avoid DSP blocks to develop the designs for resource constrained FPGAs where DSPs are very limited in number. For uniform comparison, we do not use DSPs on ZCU104 even though it has enough DSPs. The data in Table II indicates that Designs 1 (no DSPs) and 2 use LUTs for implementing multiplication and addition operations in each PE instead of the DSP blocks. From the table, we observe that Design 2 uses fewer resources than the other designs because it includes only multiplication in its PEs. The HLS synthesized designs use a large number of DSPs and $8 \times$ the LUT resources compared to the systolic implementations (Design 1, 2). It is important to note that the resources (excluding memory) used by Designs 1, 2, and 3 do not vary with the size of the input image, unlike HLS designs 4 and 5.

We show the FPGA placed layout of the MMDA method with its different blocks in Figure 6. Among all the functional units in the design, we observe that the systolic structure occupies the most resources. Figure 7 shows the FPGA device layout of the TSA design (design 3), with each systolic array tile (SAT) highlighted using a different color.

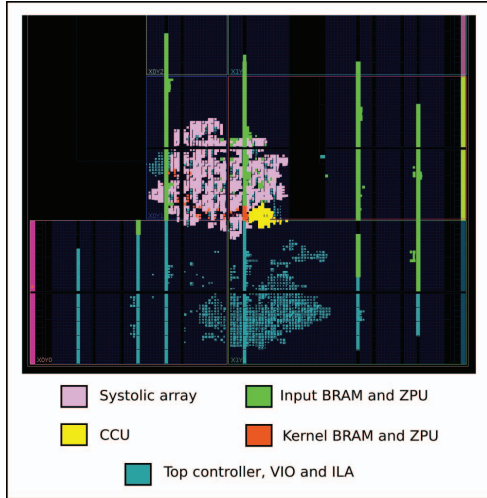


Fig. 6: Device layout showing the different units of design 1 on the FPGA on Zedboard

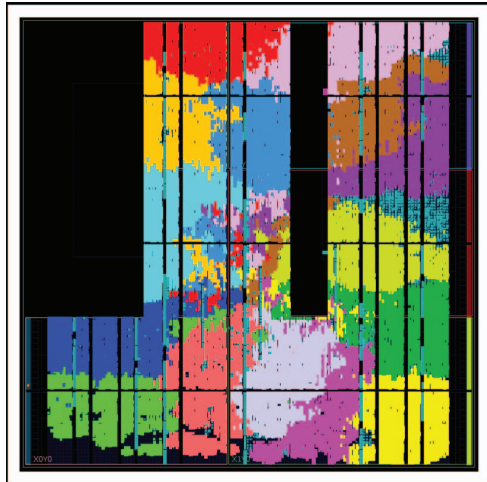


Fig. 7: Zedboard FPGA Device layout showing the 15 systolic array tiles of TSA with each tile highlighted in a different color

VI. CONCLUSION

We presented a novel methodology to perform convolution and designed a systolic architecture that aligns with the proposed methodology. We demonstrated the advantage of the proposed convolution approach on several representative image sizes from models such as LeNet-5, AlexNet, VGG-16, and Resnet-34. We observed that the proposed architecture performs $2\times$ and $2.1\times$ better than the systolic array based baseline convolution method and HLS pipelined design, respectively. We also constrained our implementation to avoid DSP blocks and use the LUTs instead, so that such designs can be implemented successfully on resource constrained FPGAs. Although the LUT utilisation increased by $9\times$, it is nearly $8\times$ fewer than the HLS pipelined implementation. Finally, we

accelerated the convolution operation by $11\times$ by generating a tiled systolic architecture that utilised all the hardware resources in the FPGA.

VII. ACKNOWLEDGEMENT

This work is supported through grant received from Science and Engineering Research Board (SERB), a statutory body of the Department of Science and Technology (DST), Government of India, under SPR/2021/000856.

REFERENCES

- [1] L. Jia, L. Lu, X. Wei and Y. Liang, "Generating Systolic Array Accelerators With Reusable Blocks," in *IEEE Micro*, vol. 40, no. 4, pp. 85-92, 1 July-Aug. 2020.
- [2] D. J. M. Moss et al., "High performance binary neural networks on the Xeon+FPGA™ platform," 2017 27th International Conference on Field Programmable Logic and Applications (FPL), 2017, pp. 1-4.
- [3] Xuechao Wei et al., "Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs," 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), 2017, pp. 1-6.
- [4] S. Kala, B. R. Jose, J. Mathew and S. Nalesh, "High-Performance CNN Accelerator on FPGA Using Unified Winograd-GEMM Architecture," in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2816-2828, Dec. 2019.
- [5] Huimin Liet al, "A high performance FPGA-based accelerator for large-scale convolutional neural networks," 2016 26th International Conference on Field Programmable Logic and Applications (FPL), 2016.
- [6] Chen Zhang et al, Optimizing FPGA-based Accelerator Design for Deep Convolutional Neural Networks. In *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*.
- [7] A. S., S. Sinha and S. K.G., "Optimization of Convolutional Neural Networks on Resource Constrained Devices," 2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI), 2019, pp. 19-24.
- [8] Z. Li et al., "Laius: An 8-Bit Fixed-Point CNN Hardware Inference Engine," 2017 IEEE International Symposium on Parallel and Distributed Processing with Applications, 2017, pp. 143-150.
- [9] Alwani, Manoj. Chen, Han. Ferdman, Michael. Milder, Peter. (2016). Fused-layer CNN accelerators. 1-12. 10.1109/MICRO.2016.7783725.
- [10] Shen, Yongming. Ferdman, Michael. Milder, Peter. (2016). Maximizing CNN Accelerator Efficiency Through Resource Partitioning. *ACM SIGARCH Computer Architecture News*. 45.
- [11] A. Podili, C. Zhang and V. Prasanna, "Fast and efficient implementation of Convolutional Neural Networks on FPGA," 2017 IEEE 28th International Conference on Application-specific Systems, Architectures and Processors (ASAP), 2017, pp. 11-18.
- [12] Z. Liu, P. N. Whatmough and M. Mattina, "Systolic Tensor Array: An Efficient Structured-Sparse GEMM Accelerator for Mobile CNN Inference," in *IEEE Computer Architecture Letters*, vol. 19, no. 1, pp. 34-37, 1 Jan.-June 2020.
- [13] G. Zhou, J. Zhou and H. Lin, "Research on NVIDIA Deep Learning Accelerator," 2018 12th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID), 2018, pp. 192-195.
- [14] Genc, Hasan. Haj Ali, et al, (2019). Gemini: An Agile Systolic Array Generator Enabling Systematic Evaluations of Deep-Learning Architectures.
- [15] Xuechao Wei et al., "Automated systolic array architecture synthesis for high throughput CNN inference on FPGAs," 2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC), 2017, pp. 1-6.
- [16] Chen, Yu-Hsin. Yang, Tien-Ju. Emer, Joel. Sze, Vivienne. (2019). Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*. 9. 292-308.
- [17] Zhang, Chen, et al. "Optimizing fpga-based accelerator design for deep convolutional neural networks." *Proceedings of the 2015 ACM/SIGDA international symposium on field-programmable gate arrays*. 2015.